

Relazione Progetto CCAI

Vincenzo Rubulotta, Paola Pappalardo, Lorenzo Varsallona

1 Introduzione e Scenario

Il presente documento descrive la progettazione, l'implementazione e la valutazione di un sistema di Intelligenza Artificiale basato su agenti, sviluppato per il corso CCAI. Il progetto mira a realizzare un sistema agentico calato nello scenario di supporto per un blogger.

Per soddisfare questo requisito, abbiamo scelto di focalizzarci sul dominio gastronomico, implementando un sistema multi-agente di supporto per un blog di cucina intelligente. L'obiettivo dell'agente è generare autonomamente ricette e contenuti culinari sempre nuovi, garantendo un'alta qualità editoriale ed evitando la ripetizione di argomenti o piatti già proposti di recente tramite l'utilizzo di un dataset vettoriale e un knowledge graph popolato dinamicamente.

L'architettura del sistema integra organicamente le tre competenze fondamentali richieste: il ragionamento agentico, l'utilizzo di una componente semantica K-RAG (Knowledge Graph-augmented RAG) e il fine-tuning di un modello linguistico di grandi dimensioni (LLM). La base di conoscenza combina in modo sinergico un Knowledge Graph editoriale per la gestione dei dati strutturati e un database vettoriale per il recupero di contenuti non strutturati, permettendo al sistema di espandere le query e validare le informazioni estratte.

1.1 Tecnologie Utilizzate

Lo sviluppo dell'intero ecosistema ha richiesto l'integrazione di diversi framework, database e modelli, selezionati per garantire modularità, scalabilità e prestazioni elevate:

- **LangGraph:** Framework di orchestrazione principale utilizzato per definire e gestire il flusso di lavoro multi-agente attraverso un'architettura a grafo direzionato, regolata da nodi e archi condizionali.
- **Neo4j:** Database a grafi nativo impiegato per ospitare la struttura semantica del Knowledge Graph editoriale, consentendo la memorizzazione e l'interrogazione delle relazioni e delle triple estratte dalle ricette.
- **ChromaDB:** Database vettoriale open-source utilizzato come memoria locale per l'indicizzazione dei documenti e per abilitare le funzionalità di recupero semantico dense della componente RAG.
- **Cohere:** Suite di modelli integrata nella pipeline di recupero per eseguire operazioni di Reranking avanzato, ottimizzando la pertinenza dei contesti estratti prima di sottoporli alla generazione.
- **Groq e Llama 3.1:70b:** Piattaforma di inferenza ad altissima velocità utilizzata per servire il modello open-weight Llama 3.1 da 70 miliardi di parametri, impiegato come motore di ragionamento logico primario dell'architettura.
- **Dataset ISMEA:** Base di dati istituzionale integrata nel sistema per estrarre informazioni strutturate sui prezzi di mercato dei prodotti agroalimentari, tracciare dinamiche economiche e suggerire alternative biologiche.

- **Dataset Custom (Giallozafferano):** Base di conoscenza documentale creata ad hoc per il progetto tramite attività di web scraping sul portale gastronomico Giallozafferano, contenente un ampio ricettario testuale esteso utilizzato per alimentare il database vettoriale.

1.2 Struttura del Grafo Multi-Agente

Il flusso di lavoro dell'agente, dalla pianificazione editoriale alla fase di stesura e revisione dei post (Human-in-the-loop), è stato modellato attraverso una serie di nodi e archi condizionali. In questo processo, l'agente mantiene in memoria lo stato interno a ogni step (ragionamento, pianificazione e output) e giustifica le proprie scelte in modo dinamico.

La Figura 1 illustra l'architettura a nodi del sistema implementato.

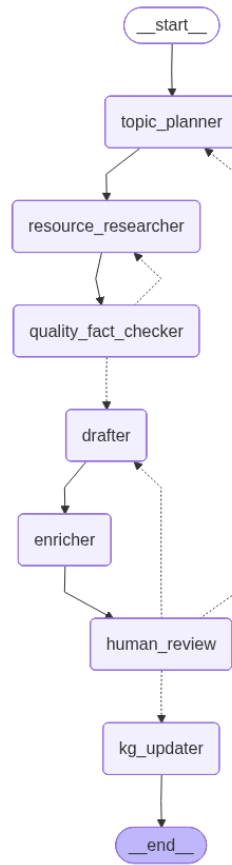


Figure 1: Architettura a nodi del sistema multi-agente per il blog di cucina.

2 Tool a Supporto dell'Agente e flusso dei nodi

Per portare a termine le proprie attività, l'architettura sfrutta il ragionamento dinamico (es. ReAct) per selezionare ed eseguire un set di strumenti in base al task corrente.

Oltre ai tool fondamentali per l'esplorazione, il recupero e l'aggiornamento della base di conoscenza, l'ecosistema del nostro agente è stato arricchito con moduli personalizzati e verticalizzati sul dominio culinario ed editoriale. Di seguito viene presentata una panoramica sintetica degli strumenti a disposizione del sistema:

- **kg_rag_tool:** Gestisce il recupero avanzato del contesto combinando dati strutturati e testi estesi attraverso una pipeline di Hybrid Search (Dense + BM25), fusione RRF e Cohere Reranking.

- **valuta_documento_locale (Fact Checker Tool):** Un modulo di controllo critico incaricato di analizzare semanticamente i testi delle ricette estratte localmente, validandone la reale pertinenza rispetto al *target topic* richiesto.
- **cerca_e_leggi_sul_web (Web Tool):** Consente l'estrazione e lo scraping multi-pagina dal web per raccogliere informazioni esterne aggiornate, garantendo un controllo incrociato e la verifica dell'accuratezza delle fonti.
- **cerca_nel_database_ismea:** Strumento custom focalizzato sulla dimensione economica e gastronomica della ricetta; interroga i dati sui prezzi e sulle alternative biologiche, suggerendo i vini ideali grazie a un LLM interno addetto al ragionamento e alla gestione dei dialetti locali.
- **aggiorna_knowledge_graph_db:** Assicura l'aggiornamento incrementale della base di conoscenza ibrida K-RAG. Salva i documenti vettorializzati nel database vettoriale e sfrutta un LLM per estrarre dinamicamente relazioni fondamentali (triple), fonti e topic, popolando strutturalmente il Knowledge Graph.

Il funzionamento dettagliato, le interazioni e le logiche interne di ciascun tool verranno approfonditi nei prossimi paragrafi.

3 Il Nodo Resource Researcher e i Tool di Ricerca

Il nodo **resource_researcher** rappresenta il nodo che si occupa di ricercare e assemblare le informazioni necessarie alla stesura del post. Sfruttando il paradigma ReAct (Reasoning and Acting), il nodo non si limita a eseguire query in modo statico, ma orchestra una vera e propria catena di pensieri (*Thought*) e azioni (*Action*) per recuperare e assemblare le informazioni necessarie alla stesura del post.

Il flusso logico imposto a questo nodo prevede una gerarchia rigida di esecuzione: la priorità assoluta è data all'interrogazione della base di conoscenza locale ibrida. Tuttavia, se i risultati interni risultano insoddisfacenti, incompleti o parziali, l'agente è istruito per comprendere le proprie lacune ed esplorare autonomamente il web per colmarle. Alla fine del processo, il nodo sintetizza le informazioni raccolte garantendo l'inserimento della fonte (URL locale o web).

Per svolgere il suo compito, il nodo ha a disposizione tre tool specifici, eseguiti in sequenza e descritti nei seguenti sottoparagrafi.

3.1 Il Tool KG-RAG (kg_rag_tool)

Il **kg_rag_tool** rappresenta il fulcro del sistema di reperimento delle informazioni, incaricato di fornire all'agente un contesto culinario robusto e completo a partire da un determinato *topic* (il nome di una ricetta). Questo strumento implementa un approccio di Hybrid Search avanzato, unendo i paradigmi del recupero strutturato (Knowledge Graph) e non strutturato (Vector Database), ottimizzati attraverso tecniche di fusione e Reranking.

Il funzionamento del tool si articola in quattro fasi principali:

1. **Interrogazione del Graph RAG (Neo4j):** Il processo inizia con una query Cypher sul database a grafi. Il sistema ricerca nodi di tipo **Recipe** che presentano una corrispondenza testuale con il topic richiesto. Se viene trovata una corrispondenza, il tool estrae le relazioni collegate, come gli ingredienti (**USES_INGREDIENT**) e le tecniche di preparazione (**USES_TECHNIQUE**).
2. **Espansione della Query:** I metadati strutturati recuperati da Neo4j (ingredienti e tecniche) non vengono semplicemente restituiti, ma sono utilizzati per arricchire dinamicamente la query originale. Ad esempio, una query base come "Risotto" viene espansa in

”Risotto Riso Carnaroli, Zafferano, Brodo Mantecatura”, aumentando significativamente il potenziale semantico per la successiva fase di recupero documentale.

3. **Recupero Ibrido (Hybrid Search):** L’interrogazione del corpus testuale (le ricette estratte da Giallozafferano) avviene su due binari paralleli:

- *Ricerca Semantica (Dense Retrieval):* La query espansa viene inviata a ChromaDB, che restituisce i 20 documenti vettorialmente più vicini nello spazio latente, catturando il significato profondo del testo.
- *Ricerca Lessicale (Sparse/Keyword Retrieval):* In parallelo, viene applicato l’algoritmo probabilistico **BM25kapi** sull’intero corpus tokenizzato, recuperando i 20 documenti con la maggiore rilevanza basata sulla frequenza esatta dei termini.

4. **Fusione e Reranking:** I risultati ottenuti dalle due ricerche sono tipicamente disomogenei (ChromaDB usa metriche di distanza, BM25 usa punteggi di rilevanza). Per combinarli efficacemente, il tool applica l’algoritmo **RRF (Reciprocal Rank Fusion)**, che normalizza e unisce le liste basandosi sulle posizioni in classifica (rank). Infine, i documenti fusi vengono passati a un modello di **Cohere Reranker**, che ricalcola lo score di pertinenza dei contesti rispetto alla query espansa, restituendo i 10 documenti più rilevanti.

Per evitare di sovraccaricare la memoria a breve termine (il contesto) dell’LLM orchestratore, il tool non restituisce direttamente i testi completi delle ricette. Al contrario, restituisce un *payload leggero* in formato JSON contenente gli ID dei documenti recuperati, i metadati strutturati (ingredienti e tecniche) e le URL delle fonti. Sarà poi compito di un tool secondario (*valuta_documento_locale*) recuperare i testi effettivi tramite i rispettivi ID ed eseguire un controllo fattuale (*Fact-Checking*) sulla loro pertinenza.

3.2 Il Tool di Validazione Locale (*valuta_documento_locale*)

Questo strumento agisce come un *Fact Checker* interno. Una volta che il *kg_rag_tool* ha individuato i documenti potenzialmente utili, il *valuta_documento_locale* interviene per analizzarli in modo critico.

Ricevendo in input il topic richiesto, gli ID dei documenti estratti e le regole fisse del Knowledge Graph, il tool interroga il modello LLM affinché valuti la reale pertinenza di ogni singolo frammento vettoriale rispetto alla ricetta desiderata. La valutazione segue regole tassative, con un controllo bloccante sul titolo originale della ricetta per evitare allucinazioni (es. scambiare la ricetta dei ”Cannoli” per delle ”Trecine” solo perché condividono alcuni ingredienti come la ricotta).

L’esito della valutazione per ogni documento può assumere tre valori:

- **SI:** Il frammento è pertinente e appartiene alla variante richiesta.
- **PARZIALE:** Il frammento descrive la ricetta base, ma manca palesemente della variante specifica richiesta dal topic (es. si richiede ”Cannoli al pistacchio” ma si trova solo ”Cannoli alla ricotta”).
- **NO:** Il frammento è irrilevante o decontestualizzato.

L’esito di questo tool rappresenta la decisione successiva dell’invocazione di eventuali tool di supporto da parte del nodo *resource_researcher*: se la valutazione restituisce documenti interamente pertinenti (”SI”), l’agente si ferma e formula la ricetta utilizzando le informazioni disponibili nei due database. Al contrario, se l’esito è ”NO” o ”PARZIALE”, il nodo acquisisce consapevolezza delle lacune nel database locale e innesca automaticamente il tool di ricerca web per arricchire il contesto.

3.3 Il Tool di Ricerca Web (*cerca_e_leggi_sul_web*)

Questo strumento entra in gioco come meccanismo di *fallback* e integrazione intelligente. Richiamato dal nodo `resource_researcher` in caso di insufficienza dei dati locali, permette all'agente di interrogare dinamicamente il web (tramite motore di ricerca DuckDuckGo) ed estrarre il contenuto testuale principale (*main content*) da più pagine contemporaneamente per permetterne il confronto.

Per garantire un'elevata qualità e affidabilità delle informazioni raccolte, scongiurando l'estrazione di testi rumorosi o decontestualizzati, il codice del tool implementa diverse logiche di controllo:

- **Blacklisting dei Domini:** Il sistema esclude a priori i risultati provenienti da enciclopedie generaliste (es. Wikipedia), social network (Facebook, Instagram, TikTok, YouTube) e piattaforme di e-commerce (Amazon, eBay). Questo filtro dirige l'agente verso fonti più autorevoli, come blog o testate specializzate in cucina.
- **Scraping e Pulizia del Testo:** Utilizza la libreria `trafilatura` per scaricare la pagina HTML e isolare esclusivamente il testo rilevante della ricetta. Il tool rimuove automaticamente il rumore di fondo, come i menu di navigazione, i banner e i commenti degli utenti.
- **Validazione Semantica e Strutturale:** Un documento estratto viene considerato valido per l'agente solo se supera una lunghezza minima di 500 caratteri e se contiene parole chiave tipiche del dominio gastronomico italiano (es. le radici "ingredient", "preparazion", "procediment", "dosi").
- **Raffinamento Iterativo della Query:** Qualora la ricerca iniziale non dovesse produrre risultati soddisfacenti, il tool esegue in automatico fino a tre tentativi. Dal secondo tentativo in poi, la query originale viene arricchita con termini forzati (es. "*titolo ricetta procedimenti dosi ingredienti*") per restringere in modo aggressivo i risultati ai soli articoli procedurali.

Una volta individuate fino a 3 opzioni valide, il tool formatta i testi accodandoli in un'unica stringa strutturata e includendo esplicitamente il tag `URL_FONTE` per ogni pagina. Questo dettaglio è cruciale per permettere all'LLM orchestratore di mantenere la tracciabilità delle risorse e soddisfare i requisiti del progetto riguardanti il *grounding* e la citazione delle fonti web.

4 Il Nodo Quality & Fact Checker

Successivamente all'esecuzione del `resource_researcher`, il flusso del grafo passa al nodo `quality_fact_checker`. A differenza dei nodi precedenti, questo componente non richiama tool per la ricerca di nuove informazioni, ma agisce come un revisore editoriale interno (Gatekeeper). Il suo obiettivo è garantire che il contenuto generato mantenga un alto standard e rispetti le logiche di pubblicazione del blog, affermandolo come una fonte di informazioni affidabile.

Il nodo prende in input le risorse grezze appena assemblate (`raw_resources`) e il topic corrente, procedendo analiticamente in due fasi distinte:

1. **Recupero dello Storico Editoriale:** Il nodo esegue una query Cypher sul database a grafi (Neo4j) per ottenere la lista completa degli argomenti (`Topic`) già trattati e pubblicati in precedenza. Questa operazione di memoria a lungo termine è fondamentale per soddisfare i requisiti di pianificazione dell'architettura agentica, assicurando la copertura di nuovi argomenti ed evitando la ridondanza di contenuti già generati in passato.
2. **Validazione tramite LLM:** Il modello linguistico viene invocato assumendo il ruolo esplicito di *Fact Checker*. Fornendogli in input il testo della bozza appena stilata e lo

storico dei topic, il sistema richiede all’LLM di rispondere in modo binario (SI/NO) a due controlli incrociati:

- *Controllo di Novità:* Verifica se la ricetta proposta è un duplicato esatto di un contenuto già presente nel Knowledge Graph.
- *Controllo Procedurale:* Valuta se i passaggi descritti espongono una preparazione coerente e proceduralmente valida per il piatto in questione.

Gestione delle Varianti e Tolleranza Sintattica

Un aspetto critico gestito all’interno del prompt di validazione riguarda la risoluzione dei falsi positivi sulle nomenclature simili. L’LLM è specificamente istruito ad accettare e validare (rispondendo ”SI”) ricette i cui nomi presentano differenze minime o preposizionali rispetto allo storico (ad esempio, ”Cannolo alla ricotta” a fronte di un preesistente ”Cannolo al pistacchio”). Questo grado di flessibilità semantica permette al sistema di non scartare varianti culinarie perfettamente valide a causa di rigide regole di *exact match*.

Se la valutazione complessiva dell’LLM ha esito positivo, le risorse vengono aggiunte alla lista dei `valid_links`. Lo stato dell’agente viene quindi aggiornato a `fact_checking_done`, consentendo all’architettura di procedere verso le fasi finali di stesura formale del post.

5 Il Nodo Drafter

Superata con successo la fase di fact-checking, il controllo del flusso passa al nodo **drafter**. Questo componente rappresenta il motore generativo dell’architettura agentica, incaricato di sintetizzare tutte le informazioni raccolte e redigere la bozza dell’articolo da sottoporre alla revisione dell’utente.

A differenza dei nodi precedenti focalizzati sulla ricerca, il **drafter** opera un’avanzata sintesi contestuale, fondendo direttive editoriali, dati strutturati e frammenti testuali. Prima di invocare il modello linguistico, il nodo interroga il Knowledge Graph (Neo4j) per estrarre le relazioni e le affermazioni storiche del blog (`past_claims`). Questo passaggio garantisce che il nuovo articolo sia coerente con i contenuti precedentemente pubblicati e offra all’agente gli spunti necessari per connettere la nuova ricetta agli argomenti già trattati.

5.1 Ingegnerizzazione del Prompt e Vincoli Generativi

Per garantire che l’output rispetti i rigidi standard qualitativi del blog culinario, il prompt inviato all’LLM (Llama 3.1:70b) è stato ingegnerizzato con una serie di regole tassative:

- **Ancoraggio al Contesto (Grounding):** Il modello è obbligato a rispettare fedelmente gli ingredienti e le tecniche estratti dal *Contesto RAG Ufficiale* (database locale ibrido) e dalle risorse web verificate, mitigando drasticamente il rischio di allucinazioni.
- **Sistema di Citazioni Inline:** In piena conformità con i requisiti architetturali, l’LLM deve esplicitamente includere citazioni nel testo per dimostrare l’utilizzo delle fonti. Quando utilizza dati vincolanti del database locale, deve inserire il tag `[Fonte: Knowledge Graph]`; quando sfrutta passaggi procedurali estratti da internet, deve inserire l’URL specifico della risorsa web analizzata.
- **Formattazione e Stile:** L’articolo deve rispettare un preciso limite editoriale (650-900 parole) e presentare una lista dettagliata degli ingredienti con quantità realistiche. È stato inoltre implementato un vincolo per inibire la generazione di prezzi o valute all’interno del testo narrativo, delegando l’analisi economica ai tool successivi (integrazione ISMEA).

- **Gestione del Feedback (Human-in-the-loop):** Il prompt è strutturato per gestire dinamicamente le iterazioni di revisione. Qualora l'utente rifiuti una bozza, il nodo re-inietta nello stato il testo scartato insieme al `human_feedback`, forzando il modello a riscrivere e correggere l'output nella generazione successiva, mantenendo la coerenza strutturale.

Al termine della generazione, il nodo esegue il parsing della risposta testuale tramite espressioni regolari e manipolazione di stringhe, separando il titolo generato dal corpo effettivo dell'articolo. Infine, aggiorna lo stato globale dell'agente impostando lo status su `draft_ready` e incrementando il contatore delle revisioni, preparando il sistema all'interazione con l'utente.

6 Il Nodo Recipe Enricher e l'Integrazione Dati (ISMEA)

Una volta che il `drafter` ha redatto il corpo principale dell'articolo, il testo generato è formalmente corretto ma privo di dettagli economici e suggerimenti commerciali. Per arricchire la ricetta con questi elementi di valore, l'architettura invoca il nodo `recipe_enricher`.

Questo componente opera come un Editor Centrale. Utilizzando un agente dedicato, il nodo analizza la bozza ed estrae la lista degli ingredienti con le rispettive quantità. Successivamente, richiama in autonomia uno strumento personalizzato (`cerca_nel_database_ismea`) per ottenere i costi, le alternative ecologiche e l'abbinamento enologico.

Il nodo è istruito tramite prompt per operare in modalità *condizionale*:

- **Integrazione Prezzi (Obbligatoria):** La lista degli ingredienti viene aggiornata inserendo il prezzo calcolato tra parentesi tonde accanto a ciascuna voce, mantenendo intatte le citazioni di fonte precedentemente generate dal `drafter`.
- **Sezione BIO/Intolleranze (Condizionale):** Solo se il tool rileva alternative valide (e logicamente sensate, escludendo anomalie come il "sale BIO"), il nodo genera in coda all'articolo un box denominato "I Consigli dello Chef", suggerendo anche opzioni senza lattosio ove pertinente.
- **Sezione Vino (Condizionale):** Se viene fornito un suggerimento enologico valido, viene aggiunta in appendice la sezione "L'Abbinamento Perfetto". In caso contrario, il nodo è istruito per omettere del tutto la sezione, evitando di inventare abbinamenti inesistenti.

6.1 Il Tool: `cerca_nel_database_ismea`

Il tool `cerca_nel_database_ismea` viene chiamato per arricchire la ricetta testuale utilizzando informazioni prese dall'ente ISMEA. Ricevendo in input la lista grezza degli ingredienti, questo modulo esegue un task di ragionamento semantico e calcolo proporzionale, appoggiandosi a sua volta all'LLM interno.

Il processo si divide in tre compiti principali:

1. **Mappatura Semantica e Sinonimi:** Il catalogo ISMEA utilizza una nomenclatura commerciale destinata all'ingrosso (es. "Saccarosio" o "Frumento tenero"), spesso distante dai termini colloquiali usati nelle ricette (es. "Zucchero a velo" o "Farina Manitoba"). L'LLM agisce da traduttore semantico, collegando logicamente l'ingrediente della ricetta alla voce corretta del database, o ignorando l'ingrediente qualora non esista una corrispondenza diretta (come per gli alcolici).
2. **Conversione e Calcolo Matematico:** Questa è la fase più critica del tool. Il catalogo fornisce prezzi all'ingrosso basati su unità di misura industriali (Tonnellate, Quintali, lotti da 100 pezzi). Il sistema applica regole di conversione tassative:

- Se l'unità è in Tonnellate (t), divide il prezzo per 1.000.000 per ottenere il costo al grammo, moltiplicandolo poi per i grammi esatti richiesti dalla ricetta.
- Se l'unità è in Quintali (100 kg), divide per 100.000.
- Se la misurazione è a pezzi (es. le uova vendute in lotti da 100), calcola il prezzo unitario per poi moltiplicarlo per il numero richiesto.

Il risultato è formattato in una stringa standardizzata che riporta il costo proporzionale esatto alla porzione (es. "Circa 0.44 € per 3 uova").

3. **Elaborazione del JSON di Output:** Alla fine del processo, il tool restituisce al nodo arricchitore una struttura JSON rigorosa e senza formattazione Markdown, contenente i prezzi calcolati per il catalogo standard, i costi delle eventuali varianti BIO e il suggerimento motivato per il vino.

7 Il Nodo Human Review: Interfaccia e Controllo Human-in-the-Loop

L'architettura agentica, per quanto autonoma nella ricerca e nella stesura, non pubblica direttamente i contenuti. Il nodo `human_review` implementa il meccanismo di revisione "Human-in-the-loop". Questo componente sospende temporaneamente l'esecuzione in background del grafo e presenta il post generato all'utente in un ambiente grafico dedicato.

Per la realizzazione dell'interfaccia utente è stata impiegata la libreria standard `tkinter`. La finestra generata espone in modo chiaro l'argomento trattato, la giustificazione editoriale che ha guidato la scelta del topic e la bozza completa e arricchita del post.

L'interfaccia permette all'utente di approvare, modificare o rifiutare e rigenerare il contenuto, offrendo tre percorsi decisionali distinti mappati su altrettanti pulsanti:

- **Approva e Pubblica (`on_approve`):** Se l'articolo soddisfa gli standard richiesti, l'utente approva la bozza. Lo stato globale viene aggiornato ad `approved`. Questa è l'unica azione che sblocca i nodi successivi, poiché l'aggiornamento del Knowledge Graph deve avvenire esclusivamente dopo l'approvazione dell'utente.
- **Riscrivi tramite Feedback (`on_rewrite`):** Se il contenuto è valido ma necessita di aggiustamenti di tono, lunghezza o stile, l'utente può inserire un feedback testuale esplicito (es. "Rendi il tono più formale" o "Rimuovi la citazione iniziale") tramite l'apposito campo di testo. Il nodo imposta lo stato su `rewrite`, rimandando il flusso al nodo `drafter`, che re-ingegnerizzerà il prompt tenendo conto della critica per rigenerare una nuova bozza.
- **Scarta Notizia (`on_reject`):** Se l'argomento generato non risulta convincente o la bozza è irrecuperabile, l'utente può rifiutare del tutto la notizia. Il nodo intercetta questa azione, aggiunge il topic alla lista dei `rejected_topics` e manipola la coda di pianificazione (`planning_queue`), permettendo all'agente di scartare il ramo corrente e ricominciare il ciclo editoriale con una nuova ricetta.

8 Il Nodo KG Updater e il consolidamento del Knowledge Graph

Se l'interazione con l'utente nel nodo `human_review` si conclude con l'approvazione del post (`status = "approved"`), il flusso dell'agente imbocca il suo ultimo passaggio: il nodo `kg_updater`. Questo componente è il responsabile finale dell'architettura K-RAG, in quanto consolida le nuove informazioni, garantendo che il sistema si evolva e incrementi la propria base di conoscenza dopo ogni ciclo editoriale concluso con successo.

Il nodo `kg_updater` agisce tramite un agente con capacità di ragionamento (ReAct). Riceve in input il titolo definitivo del post, il testo approvato (`draft`) e gli URL delle fonti validate, per poi orchestrare autonomamente il tool `aggiorna_knowledge_graph_db`.

8.1 Il Tool di Aggiornamento: `aggiorna_knowledge_graph_db`

Questo strumento svolge la duplice funzione di popolare sia il database vettoriale sia il database a grafi, chiudendo il ciclo architetturale in tre fasi distinte:

1. **Indicizzazione Vettoriale (ChromaDB):** Prima di manipolare la struttura a grafo, il tool genera un ID univoco incrementale per il nuovo post (es. `Post_42`). Successivamente, vettorializza l'intero testo finale approvato e lo salva in `collection_posts` su ChromaDB. Questo passaggio assicura che il contenuto del nuovo articolo diventi immediatamente disponibile per future ricerche semantiche tramite RAG.
2. **Estrazione delle Triple tramite LLM:** Per popolare il Knowledge Graph con dati strutturati, il testo del post viene sottoposto a un prompt di estrazione rigido. L'LLM, agendo come *Data Extractor*, è vincolato a estrarre relazioni unicamente nel formato `Soggetto | RELAZIONE | Oggetto`. Per prevenire la proliferazione di archi sconnessi e mantenere l'ontologia pulita, l'LLM può utilizzare esclusivamente relazioni pre-approvate (es. `USA_INGREDIENTE`, `USA_TECNICA`, `TIPO_DI_PIATTO`), avendo sempre il nome della ricetta come Soggetto.
3. **Inserimento e Relazionamento in Neo4j:** Una volta estratte le triple, il tool esegue una serie di query Cypher per persistere le informazioni:
 - Crea il nodo del `Post` e lo collega al `Topic` trattato (tramite l'arco `COVERS_TOPIC`), segnalando al sistema (in particolare al `topic_planner`) che l'argomento è stato "consumato".
 - Genera i nodi `Entity` per gli ingredienti e le tecniche, collegandoli tra loro e al post (`MENTIONS`) per tracciare con esattezza quali concetti sono stati citati all'interno di quello specifico testo.
 - Registra le fonti web utilizzate associandole al nodo del post tramite l'arco `USES_SOURCE`, mantenendo così l'integrità del grounding e la tracciabilità delle referenze trovate sul web.

8.2 Regole di Flusso Condizionale del Grafo

Il ciclo di vita dell'agente LangGraph è regolamentato da funzioni di routing condizionale che determinano la transizione tra i vari stati:

- **check_resources_quality:** Dopo la fase di ricerca, questo router valuta se l'agente è riuscito a raccogliere `verified_resources`. Se ci sono fonti valide, il flusso procede al `drafter` per la stesura; in caso di fallimento o fonti scartate dal *Fact Checker*, l'agente viene costretto a ripetere il ciclo di ricerca passando nuovamente per il `resource_researcher`.
- **check_approval:** Gestisce l'esito della finestra grafica di revisione. Se l'utente approva, il flusso si dirige verso l'aggiornamento del database (`kg_updater`). Se l'utente richiede una revisione strutturale inserendo un feedback, il router reindirizza il flusso al `drafter`. Infine, se la bozza viene rifiutata categoricamente, il ciclo si azzera tornando al `topic_planner` per selezionare una ricetta alternativa.